



Google Summer of Code
2026

APACHE

HUGEGRAPH

GSOC 2026

Query Engine Upgrade & Adaptation

Google Summer of Code 2026 — Project Proposal

Java 17

TinkerPop 3.5.1->3.8

Groovy 4

~350 Hours

15 Weeks

Large Project

/// TABLE OF CONTENTS

Sl. No.	Title	PAGE NO>
1.	Project and Student Details	3
2.	About Me	4
3.	Community Contributions	5 - 6
4.	Project Background and Objectives	7 – 12
5.	Implementation plan	13 - 21
6.	Project Timeline	22 - 23

// project and student details

Project Information

Name : HugeGraph
Query Engine
Upgrade &
Adaptation

Project Size : ~350 hours
(large)

Mentor : Imba Jin

Mentor Mail : jin@apache.org

STUDENT DETAILS

FULL Name : Himanshu Verma

Email : [Student Email](#)

GitHub : [BitFlicker64](#)

LinkedIn : [LinkedIn URL](#)

Twitter/X : [Bitflicker64](#)

PERSONAL INFO

Location : Delhi (NCR),
India

Time Zone : IST (UTC
+05:30)

Language : English (spoken
& written)

Phone : 8101016861

ACADEMIC INFO

Institution : Rishihood
University

Program : Bachelors in
technology

Stage : Freshman

Graduation : Expected June
2029

// Personal Project

rate-limiting-api

**JWT auth + Token Bucket rate limiting + Redis caching with eviction
correctness**

Java 21

Spring Boot 3

[GitHub Repository](#)

Redis

Docker

[Live demo](#)

About Me

Hi, I'm Himanshu Verma, a student from India passionate about open source and building systems that last. I started coding at 14, roughly 4 years ago, and the way it started was unglamorous: I wrote a switch-case chain inside a loop, looked at it, and thought wait, how does AI actually do this? That one question sent me down a rabbit hole I haven't climbed out of since.

Around the same time I had a laptop sitting around, downloaded Ubuntu, and spent weeks trying every popular Linux distro I could find. That's when open source hit me. I looked at Linux and thought: thousands of people built this, for free, and just gave it away. That was the coolest thing I had ever seen.

I completed my school CS curriculum early and built a strong foundation in Java. I primarily work in Unix environments and have been using open source tools like Git and Linux in my daily workflow for the past 3+ years. My knowledge base now includes Java, C, C++, Python, and JavaScript, and I am aspiring to be tech stack agnostic. If it compiles, I can work with it.

While planning an ERP for cold chain logistics SMEs, I realized that tracing items through warehouses and transit routes creates a massive web of interconnected data. Traditional Apache projects like OFBiz couldn't efficiently map those relationships, which led me on a deep dive looking for a highly scalable graph database. That's how I found HugeGraph. Building that ERP remains my long-term goal after GSoC, but solving its core data problem is what introduced me to this project.

Hacktoberfest was the turning point from using open source to contributing to it. I found it online, decided to try it, and what I got out of it wasn't just merged PRs. It was the discipline of writing clean commits, handling review feedback, and making changes without breaking what already works. Since then I've been contributing consistently, most notably to Apache HugeGraph, where I've submitted 13 pull requests with 11 merged across the core server and hugegraph-doc repositories.

Beyond coding, I've been running marathons since 10th standard, six so far, and spent time as a competitive powerlifter. My friends say I don't sleep much, which is mostly true. I'm here on a financial loan and I intend to make every hour of it count. I am not here for the stipend alone; I am here because I want to make a lasting impact on projects I believe in.

I am available to commit the full 15+ weeks to this project during the GSoC period, balancing academic work with consistent daily coding practice. I am proactive, I ask questions when stuck, and I document my work clearly so others can understand and build upon it.

Programming Languages

Java · C · C++ · Python · JavaScript

Tools & Platforms

Git · GitHub · Docker · Linux (Ubuntu)

Areas of Interest

Graph databases · Systems modernization · Backend · Distributed systems

Community Contributions

Open source pull requests submitted to Apache HugeGraph repositories

// APACHE / HUGEGRAPH

HG

apache / hugegraph

Core graph database server, PD, and store modules

github.com/apache/hugegraph

PR	TITLE	TYPE	STATUS
#2952	refactor(docker): migrate single-node compose from host to bridge networking	refactor	✓ Merged
#2961	fix(pd): add timeout and null-safety to getLeaderGrpcAddress()	Fix	✓ Merged
#2962	fix(pd): resolve hostname entries in IpAuthHandler allowlist	Fix	✓ Merged
#2976	chore(docker): remove tmp volume mounts after image update	Fix	✓ Merged
#2944	refactor(server): unify URL configs when scheme is missing	refactor	✓ Merged
#2941	fix(consumers): prevent await deadlock on ContextCallable failure	Fix	✓ Merged
#2949	doc: remove references to removed hugegraph-style.xml	doc	✓ Merged
#2925	doc: fix Cypher documentation link in README	doc	✓ Merged
#2963	docs: update Docker deployment docs for bridge networking migration	doc	✓ Merged
#2926	docs: add repository structure overview	doc	✕ Closed



apache / hugegraph-doc
Official documentation and deployment guides

github.com/apache/hugegraph-doc

PR	TITLE	TYPE	STATUS
#452	Harden documentation link validation to prevent false CI passes	Fix	✓ Merged
#455	doc: add docker-compose guide and update deployment docs	doc	● In Review
#449	refactor: unify URL prefix in 1.8.0	refactor	🕒 Draft
#450	fix: update links in documentation for quick navigation	Fix	✓ Merged

11

🔗 Merged

1

● In Review

1

🕒 Draft

13

TOTAL PRs

Project Title

[HugeGraph] Query Engine Modernization: Java 17 Migration, TinkerPop Upgrade, and Groovy 4 Sandbox Refactor

Background and Problem Context:

The project aims to modernize the HugeGraph query engine by migrating its core runtime stack from Java 11, TinkerPop 3.5.x, and Groovy 3 to a contemporary platform based on Java 17, TinkerPop 3.7 or 3.8, and Groovy 4. This effort focuses on resolving structural limitations introduced by dependency lag, improving runtime stability under newer JVM constraints, and redesigning the script execution security model. A central goal of the project is to replace the existing blacklist-based HugeGraphSecurity mechanism with a robust sandbox-oriented execution strategy that provides stronger guarantees against script-level vulnerabilities while reducing long-term maintenance complexity. In parallel, the migration will ensure that Gremlin traversal behavior and query semantics remain consistent across framework upgrades.

Although dependency upgrades and runtime adaptations have been addressed incrementally in prior discussions and issues within the HugeGraph community, a comprehensive query engine modernization has not yet been executed as a unified engineering effort. This project consolidates those scattered concerns into a systematic platform transition plan that addresses compatibility, security, and performance considerations together rather than in isolation. By aligning HugeGraph with modern JVM and TinkerPop ecosystems, the system benefits from improved garbage collection behavior, stricter runtime safety guarantees, and enhanced framework capabilities without compromising existing query workflows.

Beyond version alignment, the project emphasizes behavioral correctness and operational stability. Upgrading core query dependencies can introduce subtle semantic drift, traversal strategy changes, and runtime incompatibilities that directly affect query results. Therefore, the modernization process prioritizes compatibility validation, regression detection, and controlled refactoring of integration layers. The expected outcome is a query engine that is easier to maintain, safer to operate in multi-tenant environments, and better positioned for future evolution of both the JVM and TinkerPop frameworks.

OBJECTIVE 1

Core Dependency and Build Modernization

Before performing any functional or security-level enhancements, the foundational runtime and query ecosystem of HugeGraph must be stabilized on modern, supported versions. This objective focuses strictly on dependency upgrades, compatibility restoration, and build correctness without altering query logic or storage behavior. A risk assessment using the TinkerPop 3.8.0 upgrade documentation against the HugeGraph codebase identified 13 concrete breaking changes across critical and high severity — each mapping to specific files with a clear fix strategy.

● Shaded Jackson & Kryo Decoupling

The most architecturally significant change. HugeGraph uses `org.apache.tinkerpop.shaded.jackson.*` across 14+ production files covering JSON utilities, GraphSON serialization, auth models, and metrics — all rated CRITICAL. All shaded Jackson imports are replaced with `com.fasterxml.jackson.*` and a direct `jackson-databind` Maven dependency is declared. Separately, KryoUtil uses `org.apache.tinkerpop.shaded.kryo.*` for internal backend byte-level serialization entirely separate from Gryo IO; a standalone `com.esotericsoftware:kryo` dependency replaces it with persisted data compatibility verified.

● Gryo IO Migration & MetricManager Replacement

KryoUtil uses HugeGryoModule registers custom type serializers into TinkerPop's Gryo IO framework — this layer is migrated to GraphBinary after validating custom type registration feasibility, since GraphBinary does not expose the same extension points as Gryo. Note: `gremlin-server.yaml` already uses `GraphBinaryMessageSerializerV1` at the network level. Separately, `MetricManager.INSTANCE.getRegistry()` used across five files is replaced with a `MetricRegistry` singleton from `Dropwizard Metrics`, with `ServerReporter` lifecycle integration in `GraphManager.addMetrics()` preserved.

● Apache TinkerPop Upgrade (Incremental)

The upgrade from 3.5.1 to 3.8 proceeds incrementally — validating 3.6.x, then 3.7.x, then 3.8.x upgrade documentation at each step. Breaking changes addressed include: `GroovyTranslator` returning `String` directly (fixing dead `Script` variable in `HugeGraphAuthProxy.java`), `GremlinExecutor` API changes in `CypherOpProcessor` including a runtime-only reflective access failure via `Whitebox.invoke()`, `OpProcessor` API removal, `TinkerPopJacksonModule` removal, `CredentialGraphTokens` relocation, and `TraversalEngine` + `AbstractGremlinSuite` removal.

● Groovy 4 Migration

The embedded Gremlin script engine is migrated from Groovy 3 to Groovy 4. Key adjustments include replacing deprecated Groovy APIs, resolving compilation rule differences, fixing script engine initialization issues, and maintaining stable script execution under the new Groovy version. The `gremlin-groovy-test` artifact removal — currently pinned at 3.2.11, already mismatching project TinkerPop 3.5.1 — is resolved as part of this workstream.

● Toolchain & Test Framework Modernization

maven-compiler-plugin 3.1 (predates Java 9 module support entirely) is upgraded to 3.11+. maven-surefire-plugin 2.20 is upgraded to 3.1.x and explicitly pinned across all six modules including five build profiles in hugegraph-server that currently inherit silently from the Apache parent POM. hugegraph-commons (uses its own compiler.source=1.8 property) is migrated independently. All five PowerMock 2.0.0-RC.3 artifacts in hg-pd-test and hg-store-test are replaced with Mockito equivalents — Javassist-based bytecode manipulation is incompatible with the Java 17 module system.

● Dependency Alignment & Conflict Resolution

Four Jackson versions coexist (2.13.0, 2.13.2, 2.14.0-rc1, 2.15.2) with the DependencyConvergence enforcer rule explicitly commented out — unified under a single stable BOM before the Java 17 rollout. Where version realignment is insufficient, class relocation via the Maven Shade plugin prevents classpath collisions without requiring upstream library changes. RocksDB JNI at 8.10.2 in the server and 7.7.3 in the store are validated for binary compatibility with Java 17 through integration testing.

● Gremlin Traversal Compatibility

Dependency upgrades introduce subtle behavioral differences in traversal steps that must be validated against the actual upgrade documentation. The highest-risk coupling is TraversalUtil.extractOrder(), which calls orderStep.getComparators() and accesses .getValue1() on the returned javatuples Pair — a compile-time break if TinkerPop 3.8 changes the return type. The choose step enforcing first-matched-option-only logic and dateDiff shifting from seconds to milliseconds are verified before claiming regression coverage.

● Container, CI & Runtime Validation

All three Dockerfiles hardcoded to eclipse-temurin:11 and all four CI workflow files pinned to Java 11 are updated to Java 17, with the Java 8 install step in server-ci.yml for HBase retained. Production code accessing internal JDK APIs directly — sun.misc.Unsafe in OffHeapMemoryUtil.java and IntMapByDynamicHash.java, and jdk.internal.reflect in Reflection.java and HugeFactoryAuthProxy.java — is validated through explicit integration testing under Java 17, as these are runtime risks that do not surface during compilation.

// End Result

All 13 breaking changes resolved. Shaded Jackson replaced with direct com.fasterxml.jackson dependency across 14+ files. KryoUtil decoupled from TinkerPop shading. Gryo IO layer migrated to GraphBinary. MetricManager replaced with Dropwizard singleton. PowerMock removed. Jackson versions unified under a single BOM. Clean compilation under Java 17, successful server startup, no runtime linkage failures, and preserved Gremlin query behavior across all core operations.

OBJECTIVE 2

Gremlin Execution Security Redesign

The existing HugeSecurityManager detects Gremlin context through thread name prefix checks (gremlin-server-exec, task-worker) and full stack trace walks — an approach the codebase itself acknowledges as imprecise through an inline TODO. More critically, Gremlin-specific enforcement is interleaved with backend socket checks for MySQL, HBase, Cassandra, Raft, and SofaRPC inside the same checkXxx() override methods, making independent disablement impossible without refactoring each method. This objective replaces that runtime detection model with a four-layer compiler-enforced sandbox operating entirely at the Groovy compilation stage.

● Engine Factory Replacement

HugeGraph currently routes all Gremlin execution through SingleGremlinScriptEngineManager.get(), a TinkerPop-managed singleton with no injection surface. HugeGraphGremlinPlugin provides only class imports and offers no compiler configuration hook. HugeGraphGremlinPlugin is extended to register a custom script engine factory that builds a GremlinGroovyScriptEngine with an attached CompilerConfiguration — establishing the injection surface onto which all sandbox layers are wired. Configuration properties for allowed types, method allowlist, and script timeout are added to hugegraph.properties.

● SecureASTCustomizer (Layer 1)

Attached to CompilerConfiguration at engine creation. Blocks dangerous language constructs at parse time before type analysis: import statements, package declarations, class definitions, and reflection expressions. Method calls remain permitted since Gremlin traversals are expressed as method chains. This eliminates entire exploit categories — including string obfuscation and Groovy metaprogramming tricks — before type analysis even begins.

● TypeCheckingExtension (Layer 2)

Enforces an allowed package allowlist at compile time: org.apache.tinkerpop.gremlin.* and org.apache.hugegraph.* graph APIs only. Anything outside produces a compile error. Because g and graph are injected as runtime bindings in HugeScriptTraversal, compatibility between static type checking and the dynamic DSL must be validated iteratively — overly strict checking may break legitimate traversals and requires tuning.

● TimedInterruptCustomizer (Layer 3)

Injects interrupt checks directly into loops and method calls at the bytecode level during compilation, providing CPU exhaustion control at the script level rather than relying solely on the server-level evaluationTimeout wall clock. The existing evaluationTimeout remains as an outer safety net. The interaction between bytecode interrupts and async traversal execution across server-managed threads must be validated.

● GroovyValueFilter (Layer 4)

Operates after script evaluation to prevent sensitive internal objects from being returned to the caller. A script may run correctly within sandbox constraints but attempt to return an internal HugeGraph configuration object or backend store reference. The filter inspects returned values including traversal results, iterables, and nested structures, blocking anything outside the permitted return type surface — closing the post-eval leakage vector that the compile-time layers cannot address.

● HugeSecurityManager Domain Separation

A dedicated GremlinSecurityPolicy class is extracted from HugeSecurityManager, encapsulating all Gremlin-specific enforcement behind a clear class boundary. GremlinSecurityPolicy is disabled when the compile-time sandbox is active. Backend protections for MySQL, HBase, Cassandra, Raft, SofaRPC, and snapshot logic are preserved unchanged. HugeSecurityManager remains as a fallback runtime guard covering reflection edge cases, native calls, and ClassLoader behavior that static analysis cannot fully address. Java 17 deprecates SecurityManager for eventual removal, so non-Gremlin protections are also prepared for a future migration path.

// End Result

A four-layer compiler-enforced Gremlin sandbox — SecureASTCustomizer, TypeCheckingExtension, TimedInterruptCustomizer, and GroovyValueFilter — replacing the fragile runtime detection model. HugeSecurityManager Gremlin checks isolated behind GremlinSecurityPolicy and disabled when the sandbox is active. Backend protections fully preserved. Sandbox configuration externalized to hugegraph.properties without code changes.

OBJECTIVE 3

Validation, Testing, and Performance Benchmarking

With the dependency and security layers modernized, the final objective validates the complete upgraded stack end-to-end. The scope is grounded in what the repository actually contains: Spring Boot 2.5.14 in both PD and Store, gRPC 1.39.0 in PD with a confirmed Java 17 compatibility gap, TinkerPop compliance tests gated to release branches only, and no existing graph benchmark harness. All of these constraints shape the actual work.

● PD & Store Module Migration

PD and Store are migrated from Spring Boot 2.5.14 to 2.7.x — the last 2.x release with official Java 17 support. Boot 3 migration is explicitly out of scope as it requires a full Jakarta namespace shift and constitutes a separate engineering effort. gRPC in PD is upgraded from 1.39.0 to 1.42+ to resolve known internal JDK API access issues on Java 17. RocksDB JNI 7.7.3 in Store is validated for binary compatibility through integration testing. Separate RocksDB data paths are configured for the Java 11 baseline and Java 17 target environments to prevent forward-compatibility issues with stored data files.

● Test Suite Stabilization

The run-tinkerpop-test.sh script referenced in CI does not exist in the repository — TinkerPop compliance tests are effectively not running on any CI branch currently. The script is written and enabled on the Java 17 CI matrix. TinkerPop compliance tests defined via ProcessStandardTest and StructureStandardTest, activated through Maven profiles and gated to release-* or test-* branches, are restored to active execution. Unit test failures from dependency upgrades are resolved systematically, prioritizing the Gremlin execution path, CRUD operations, and serialization behavior.

● Regression Verification

Semantic changes across TinkerPop 3.6–3.8 are validated against the actual upgrade documentation rather than assumed. The confirmed highest-risk coupling is TraversalUtil.extractOrder(), which calls orderStep.getComparators() and accesses .getValue1() on the returned Javatuples Pair to extract the ElementValueComparator — a compile-time break if TinkerPop 3.8 changes the return type of getComparators(). The choose step behavioral change and dateDiff return value shift from seconds to milliseconds are cross-checked against HugeGraph's actual traversal usage before claiming regression coverage.

● Performance Benchmark Construction

The existing benchmark infrastructure is limited to a toy PerfExampleBase harness (100 edges) and the standard TinkerPop performance suite is entirely blacklisted due to a vertex-label deletion failure above 65,536 vertices — unusable at twitter-2010 scale. The benchmark infrastructure for the twitter-2010 SNAP dataset (~1.47B edges) is written as part of this phase, targeting KoutTraverser directly via /traversers/kout at depths 1 and 2. The Java 11 baseline and Java 17 target run against separate RocksDB data paths with latency, throughput, and GC pause metrics captured and compared.

// End Result

PD and Store running cleanly on Java 17 with Spring Boot 2.7.x and gRPC 1.42+. TinkerPop compliance tests restored and passing on the Java 17 CI matrix. All unit and integration tests passing. TraversalUtil OrderGlobalStep API coupling verified against TinkerPop 3.8 changelog. Twitter-14B benchmark harness built and executed against Java 11 baseline and Java 17 target with separate RocksDB data paths. Performance report produced with latency and throughput comparison.

Implementation Plan

HugeGraph Query Engine Upgrade & Adaptation

Java 17

TinkerPop 3.5.1->3.8

Groovy 4

~350 Hours

15 Weeks

PHASE 01

Dependency Analysis & Core Upgrade

The first step before touching any code is a complete audit of what will break. A risk assessment using the TinkerPop 3.8.0 upgrade documentation against the HugeGraph codebase identified 13 breaking changes. Each one maps to specific files and has a clear fix strategy.

Breaking Change	Affected Files	Severity
Shaded Jackson Removal	JsonUtil.java HugeGraphSONModule.java SchemaDriver.java RolePermission.java HugeAuthenticator.java + 4 additional files	CRITICAL
Shaded Kryo Removal (Backend)	KryoUtil.java ByteBuffer.java HugeGryoModule.java HugeGraphWriteTest.java + 4 additional files	CRITICAL
Gryo IO Framework Removed	HugeGryoModule.java HugeGraphIoRegistry.java HugeGraphWriteTest.java	HIGH
MetricManager Removed	MetricsUtil.java GraphManager.java ApplicationConfig.java AccessLogFilter.java HugeFactoryAuthProxy.java	HIGH
GroovyTranslator returns String	Script type variable at L2147 + dead Script import at L93 both fail. HugeGraphAuthProxy.java	HIGH
GremlinExecutor API Changed	CypherOpProcessor.java globalBindings via Whitebox.invoke() is runtime-only failure — harder to catch in CI.	HIGH
OpProcessor API Removed	CypherOpProcessor.java CypherOpProcessor extends AbstractEvalOpProcessor. If server processing model changed, full rewrite required.	HIGH
TinkerPopJacksonModule Removed	HugeGraphSONModule.java HugeGraphSONModule extends TinkerPopJacksonModule directly.	HIGH
gremlin-groovy-test Removed	hugegraph-test/pom.xml hugegraph-server/pom.xml Pinned at 3.2.11 — already mismatches project TinkerPop 3.5.1.	HIGH
OrderGlobalStep Pair Type Changed	TraversalUtil.java L-210 uses .getValue1() for comparator extraction. If Pair replaced by Map.Entry, accessor and cast both break. getPropertyKey() + getValueComparator() downstream also at risk.	HIGH

CredentialGraphTokens Relocated	HugeAuthenticator.java WsAndHttpBasicAuthHandler.java If constant names change, KEY_USERNAME and KEY_PASSWORD silently change — runtime auth failure risk.	MEDIUM
Auth Handler Hierarchy Changed	WsAndHttpBasicAuthHandler.java constant names change, KEY_USERNAME and KEY_PASSWORD silently change — runtime auth failure risk.	MEDIUM
TraversalEngine + AbstractGremlinSuite Removed	ProcessBasicSuite.java StructureBasicSuite.java Both constructors pass TraversalEngine.Type.STANDARD to super(). Test-only impact.	MEDIUM

● Shaded Jackson Decoupling (Critical)

The most architecturally significant change. HugeGraph uses [org.apache.tinkerpop.shaded.jackson.*](#) across 14+ production files covering JSON utilities, GraphSON serialization, auth models, and metrics. All shaded Jackson imports are replaced with [com.fasterxml.jackson.*](#) and a direct jackson-databind Maven dependency is declared. This affects JsonUtil, HugeGraphSONModule, RolePermission, HugeAuthenticator, and MetricsModule at minimum. HugeGraphSONModule also requires a replacement for its TinkerPopJacksonModule base class.

● Shaded Kryo Decoupling (Critical)

KryoUtil uses [org.apache.tinkerpop.shaded.kryo.*](#) for internal backend byte-level serialization of values stored in the backend store. This is entirely separate from Gryo IO and cannot be replaced by GraphBinary. If TinkerPop removes shaded Kryo, a standalone [com.esotericsoftware:kryo](#) dependency is added and all KryoUtil imports are rewritten against it. Persisted data compatibility is verified during this migration since KryoUtil handles backend binary format.

● Gryo IO Migration (High)

HugeGryoModule registers HugeGraph's custom type serializers into TinkerPop's Gryo IO framework using GryoIo.class as the registry key. This IO registration layer is migrated to GraphBinary. Feasibility of custom type registration in GraphBinary is validated before rewriting serializers, since GraphBinary does not expose the same extension points as Gryo. Note: gremlin-server.yaml already uses GraphBinaryMessageSerializerV1 at the network level, meaning the network layer is already migrated. The IO registry and test-time Gryo usage are the remaining scope

● MetricManager Replacement (High)

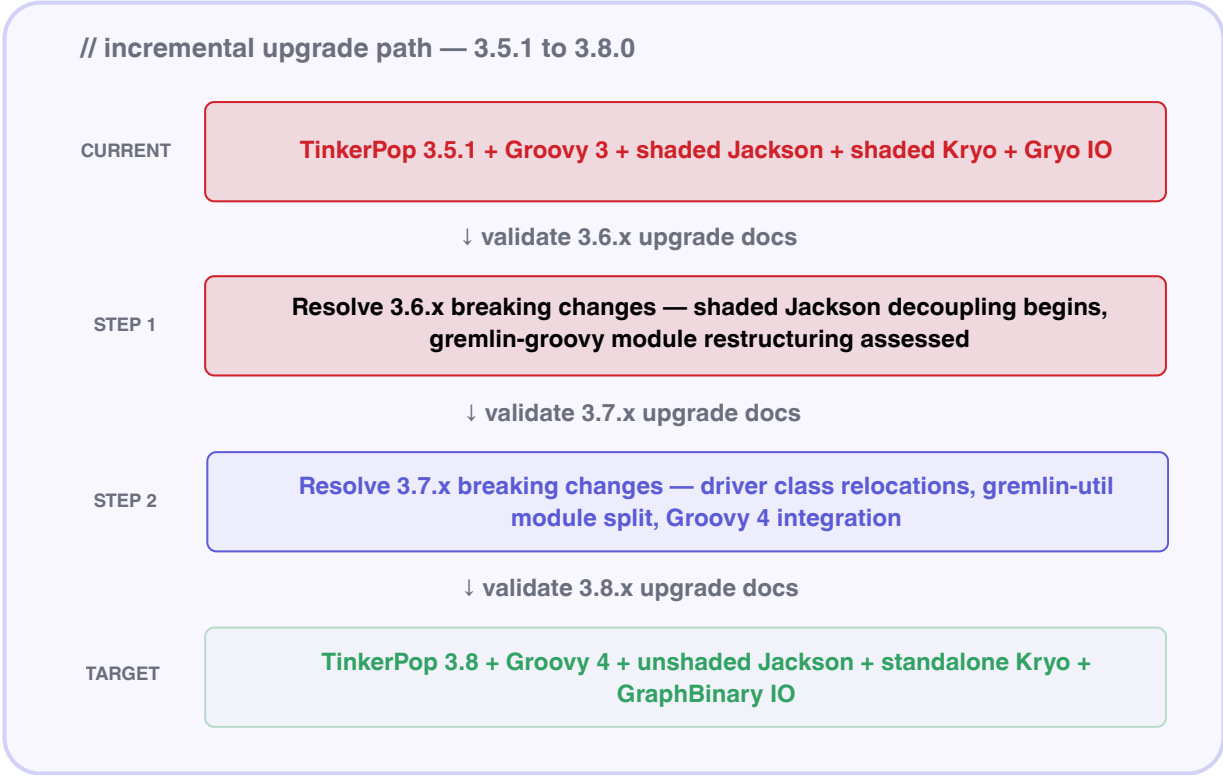
MetricManager.INSTANCE.getRegistry() is used across MetricsUtil, ApplicationConfig, GraphManager, and MetricsAPI. MetricsUtil.REGISTRY is a static final field initializer — a class-load failure here cascades to every class that references it. The replacement is a MetricRegistry singleton instantiated directly from Dropwizard Metrics. Critically, the ServerReporter lifecycle integration in GraphManager's addMetrics() method must be preserved since it hooks into server startup reporting, not just metric registration.

● GremlinExecutor and OpProcessor (High)

Two separate risks exist here. CypherOpProcessor's call to [context.getGremlinExecutor\(\).getExecutorService\(\)](#) is a compile-time break if getExecutorService() is removed. The Whitebox.invoke() reflective access to globalBindings in ContextGremlinServer is a runtime-only failure — it compiles but fails at server startup with a NoSuchFieldException if the internal field is renamed. This is reclassified as HIGH risk because reflective breakage does not surface in compilation. CypherOpProcessor's full dependency on AbstractEvalOpProcessor and the Op Processor API is also assessed for post-3.5 server model changes.

● GroovyTranslator, CredentialGraph, TraversalEngine

In HugeGraphAuthProxy.java, the Script variable at L2147 and dead Script import at L93 both fail when translate() returns String directly. Fix removes the Script variable and uses the String return directly. CredentialGraphTokens is updated to the new package path in HugeAuthenticator and WsAndHttpBasicAuthHandler — if constant names change, runtime auth failures are possible since KEY_USERNAME and KEY_PASSWORD are embedded in the public interface. TraversalEngine.Type.STANDARD references in ProcessBasicSuite and StructureBasicSuite are removed and super constructor calls updated.



Phase 01 Deliverable

All 13 identified breaking changes resolved. Shaded Jackson removed and replaced with direct `com.fasterxml.jackson` dependency across 14+ files. KryoUtil decoupled from TinkerPop shading and migrated to standalone Kryo. Gryo IO layer migrated to GraphBinary with custom type registration validated. MetricManager replaced with Dropwizard singleton with ServerReporter lifecycle preserved. Reflective GremlinExecutor access verified and documented. Clean compilation of Server module against TinkerPop 3.8 and Groovy 4.

PHASE 02

Java 17 Runtime & Environment

With the dependency layer clean, the runtime environment is migrated to Java 17. A line-by-line verification of the actual repository reveals four confirmed blockers and three high-risk areas. Critically, the `--illegal-access` flag is already absent from the codebase and the project correctly uses `--add-exports` already. The internal JDK API surface via `sun.misc.Unsafe` and `jdk.internal.reflect` is a production runtime risk that must be explicitly validated, not just the test framework issues. Migration proceeds in five structured workstreams.

Area	Verified State	Classification
maven-compiler-plugin	Version 3.1 in root pom.xml — predates Java 9 module support entirely. Cannot handle module-path compilation.	BLOCKED
maven-surefire-plugin	Version 2.20 pinned in six modules. Additionally, hugegraph-server declares surefire across five build profiles with no explicit version — root pluginManagement does not manage surefire, so these profiles silently inherit from the Apache parent POM. Must be explicitly pinned to 3.1.x everywhere or behavior under Java 17 is inconsistent across modules.	BLOCKED

hugegraph-commons compiler baseline	Uses compiler.source=1.8 — separate property name, does NOT inherit root setting. Compiles to Java 8 bytecode independently. Must be migrated as a separate unit.	BLOCKED
PowerMock 2.0.0-RC.3	Present in hg-pd-test/pom.xml and hg-store-test/pom.xml . Two artifacts explicitly declare compile scope; the remaining three use Maven default compile scope — functionally identical but not all explicitly scoped. Javassist-based bytecode manipulation is incompatible with Java 17 module system regardless of scope.	BLOCKED
Internal JDK API usage	Reflection.java and HugeFactoryAuthProxy.java access <code>jdk.internal.reflect.OffHeapMemoryUtil.java</code> and IntMapByDynamicHash.java use <code>sun.misc.Unsafe</code> via direct import. Source note: "this class may easily crash". This is a runtime risk, not just a test concern.	HIGH RISK
Jackson version fragmentation	Four versions coexist: 2.13.0 , 2.13.2 , 2.14.0-rc1 , 2.15.2 . <code>DependencyConvergence</code> rule explicitly commented out. Stability risk requiring alignment before Java 17 rollout.	HIGH RISK
Docker and CI pinning	All three Dockerfiles hardcoded to <code>eclipse-temurin:11</code> . All four CI workflows pinned to Java 11 only. Java 8 install step in <code>server-ci.yml</code> for HBase must be retained.	HIGH RISK
RocksDB JNI	Two versions in production: 8.10.2 in <code>hugegraph-server</code> , 7.7.3 in <code>hugegraph-store</code> . Binary compatibility with Java 17 runtime validated through integration testing.	VALIDATE
--add-exports flags	Declared with conditional activation — flags engage only when Java version is greater than 9, which means they activate correctly under Java 17 without modification. Preserved and revalidated under Java 17. Extended only if integration testing reveals additional access requirements beyond current declarations.	PRESERVE

// migration workstreams

TOOLCHAIN

Upgrade `maven-compiler-plugin 3.1` → `3.11+` · Update compiler source/target to 17 in root `pom.xml` · Migrate `hugegraph-commons` separately via its own `compiler.source` property · Upgrade `maven-surefire-plugin 2.20` → `3.1.x` across all six modules and explicitly pin in `hugegraph-server` build profiles

TEST FRAMEWORK

Replace all five PowerMock artifacts (2.0.0-RC.3) in `hg-pd-test` and `hg-store-test` with Mockito equivalents · Rewrite affected test classes

DEPENDENCY ALIGNMENT

Unify Jackson [2.13.0](#) / [2.13.2](#) / [2.14.0-rc1](#) / [2.15.2](#) under a single stable BOM version · Evaluate and gradually enforce `DependencyConvergence` · Validate RocksDB JNI [8.10.2](#) and [7.7.3](#) via integration testing

RUNTIME VALIDATION

Preserve all `--add-exports` flags unchanged · Validate `OffHeapMemoryUtil.java` and `IntMapByDynamicHash.java` under Java 17 · Update `MIN_JAVA_VERSION` in `hugegraph-server.sh` and `EXPECT_JDK_VERSION` in `start-hugegraph-store.sh` from 11 to 17

CONTAINER AND CI

Update all three Dockerfiles to `eclipse-temurin:17` · Update all four CI workflow files to Java 17 · Retain Java 8 install step in `server-ci.yml` for HBase · Run full test suite · Confirm clean startup

● Toolchain Modernization

The maven-compiler-plugin at version 3.1 predates Java 9 module support entirely and is upgraded to 3.11 or higher first. The maven-surefire-plugin at version 2.20 is upgraded to 3.1.x across all modules. Critically, hugegraph-server declares surefire in five build profiles without an explicit version — since root pluginManagement does not manage surefire, these profiles inherit silently from the Apache parent POM. Surefire must be explicitly pinned to 3.1.x in all profile declarations to avoid inconsistent fork behavior under Java 17. The hugegraph-commons module also requires separate handling because it uses its own compiler.source property and compiles to Java 1.8 bytecode independently of the root configuration.

● Test Framework Stabilization

PowerMock 2.0.0-RC.3 is present in both hg-pd-test and hg-store-test with five artifacts in each. Two are explicitly compile-scoped; the remaining three rely on Maven default compile scope, which is functionally identical. All five are incompatible with the Java 17 module system because the Javassist bytecode manipulation they depend on cannot operate across module boundaries. All PowerMock usages are replaced with Mockito equivalents. This is a hard prerequisite for the PD and Store test suites to execute at all under Java 17.

● Dependency Alignment

Jackson coexists at four versions with the DependencyConvergence enforcer rule explicitly commented out. Fragmented versions are a stability risk requiring alignment before the Java 17 rollout. The versions are unified under a single stable BOM. Dependency convergence enforcement is evaluated and gradually introduced to avoid breaking the build unexpectedly. RocksDB JNI at 8.10.2 in the server and 7.7.3 in the store are validated for binary compatibility with the Java 17 runtime through integration testing rather than assumed compatible.

● Runtime Validation and Container Alignment

The existing --add-exports flags are preserved and revalidated under Java 17. Their conditional activation logic already ensures they engage only when running on Java 9 or above, so no modification to the activation condition is needed — only continued necessity is verified. Production code accesses internal JDK APIs directly: sun.misc.Unsafe is used in the off-heap memory and hash map utilities, and jdk.internal.reflect is accessed in the reflection and auth proxy components. These are runtime risks that do not surface as compile failures and must be validated through explicit integration testing under Java 17. All three Dockerfiles and all four CI workflow files are then updated to Java 17 with the HBase Java 8 install step retained.

Phase 02 Deliverable

maven-compiler-plugin upgraded from 3.1. maven-surefire-plugin explicitly pinned to 3.1.x in all modules and all server build profiles — no implicit inheritance from parent POM. hugegraph-commons compiler baseline independently migrated to Java 17. PowerMock removed from PD and Store test modules. Jackson versions aligned under a single BOM. Internal JDK API usage preserved and validated under Java 17 through integration testing. All three Dockerfiles updated to eclipse-temurin:17. All CI pipelines running against Java 17. HugeGraph server starting cleanly.

PHASE 03

Security Model Refactor

ARCHITECTURAL SCOPE NOTE

HugeGraph currently has no compiler configuration hook. All Gremlin execution flows through SingleGremlinScriptEngineManager.get(), a TinkerPop-managed singleton with no injection surface. HugeGraphGremlinPlugin provides only class imports. Phase 03 therefore begins with establishing that engine control surface before any sandbox layer can be built.

The current HugeSecurityManager detects Gremlin context through a dual mechanism: it checks thread name prefixes (gremlin-server-exec, task-worker) then walks the full stack trace looking for GremlinGroovyScriptEngine in the call chain. This is structurally fragile — the codebase itself contains a TODO acknowledging the detection is imprecise. More critically, the security checks covering Gremlin execution are interleaved with checks governing backend sockets (MySQL, HBase, Cassandra), Raft operations, SofaRPC, and snapshot logic inside the same checkXxx() override methods. These cannot be disabled independently without refactoring each method. Phase 03 is therefore a two-stage redesign: first establishing engine-level control, then restructuring the security layer.

// current execution and security model — actual state

ENTRY POINT

GremlinJob.execute() or GraphManager.prepareSchema() constructs HugeScriptTraversal

ENGINE FETCH

SingleGremlinScriptEngineManager.get("gremlin-groovy") — TinkerPop singleton, no injection hook

EVAL

engine.eval(script, bindings) — g and graph bound at runtime, no pre-eval validation

SECURITY

HugeSecurityManager intercepts JVM calls at runtime — Gremlin + backend checks interleaved in same methods

RESULT

Cast to Traversal.Admin or stored as this.result — no post-eval interception layer exists

// target execution model after phase 03

ENTRY POINT

HugeScriptTraversal — unchanged interface

ENGINE FACTORY

Custom GremlinGroovyScriptEngine wrapper replacing SingleGremlinScriptEngineManager — injects CompilerConfiguration at creation

COMPILE GATE

SecureASTCustomizer + TypeCheckingExtension validate script before bytecode is produced — reject disallowed constructs and types

EXEC GUARD

TimedInterruptCustomizer bytecode injection controls CPU exhaustion — replaces wall-clock-only evaluationTimeout

SECURITY

HugeSecurityManager refactored — Gremlin checks isolated from backend checks per domain, Gremlin enforcement disabled when sandbox is active

// groovy 4 sandbox stack — build sequence

01 Engine Factory Replacement

The existing `--add-exports` flags are preserved and revalidated under Java 17. Their conditional activation logic already ensures they engage only when running on Java 9 or above, so no modification to the activation condition is needed — only continued necessity is verified. Production code accesses internal JDK APIs directly: `sun.misc.Unsafe` is used in the off-heap memory and hash map utilities, and `jdk.internal.reflect` is accessed in the reflection and auth proxy components. These are runtime risks that do not surface as compile failures and must be validated through explicit integration testing under Java 17. All three Dockerfiles and all four CI workflow files are then updated to Java 17 with the HBase Java 8 install step retained.

PREREQUISITE

02 SecureASTCustomizer

Attached to `CompilerConfiguration` at engine creation. Blocks dangerous language constructs at parse time: import statements, package declarations, class definitions, and reflection expressions. Method calls remain permitted since Gremlin traversals are expressed as method chains. This eliminates entire exploit categories before type analysis.

COMPILE TIME

03 TypeCheckingExtension

Enforces allowed package allowlist at compile time: `org.apache.tinkerpop.gremlin.*` and `org.apache.hugegraph.*` graph APIs only. Anything outside produces a compile error. Risk note: because `g` and `graph` are injected as runtime bindings in `HugeScriptTraversal`, compatibility between static type checking and the dynamic DSL must be validated. Overly strict checking may break legitimate traversals and requires iterative tuning.

COMPILE TIME

04 TimedInterruptCustomizer

Injects interrupt checks into loops and method calls at the bytecode level during compilation. Provides CPU exhaustion control at the script level rather than relying solely on the server-level `evaluationTimeout` wall clock. The existing `evaluationTimeout` remains as an outer safety net. The interaction between bytecode interrupts and async traversal execution across server-managed threads must be validated.

COMPILE TIME

● HugeSecurityManager Domain Separation

The current `checkXxx()` methods intermix Gremlin detection with backend-specific allowlists for MySQL, HBase, Cassandra, Raft, SofaRPC, and snapshot operations inside the same conditional chains. The refactoring extracts a dedicated `GremlinSecurityPolicy` class that encapsulates all Gremlin-specific enforcement decisions. `HugeSecurityManager` delegates Gremlin decisions to this class, creating a clear class boundary between Gremlin policy and backend policy. `GremlinSecurityPolicy` can be disabled independently when the compile-time sandbox is active. Backend protections covering socket access, network connections, and filesystem access for backends are preserved unchanged and continue to be enforced at runtime. Compile-time enforcement via the sandbox stack becomes the primary boundary. `HugeSecurityManager` remains as a fallback runtime guard during the transition period, covering reflection edge cases, native calls, and `ClassLoader` behavior that static analysis cannot fully address. Java 17 deprecates `SecurityManager` for removal, so the refactoring also prepares non-Gremlin protections for eventual migration.

● Engine Replacement and Sandbox Wiring

`HugeGraphGremlinPlugin` is extended to register a custom script engine factory rather than only providing class imports. The factory builds a `GremlinGroovyScriptEngine` with an attached `CompilerConfiguration` carrying the `SecureASTCustomizer`, `TypeCheckingExtension`, and `TimedInterruptCustomizer`. Configuration properties for allowed types, method allowlist, and script timeout are added to `hugegraph.properties`. `TypeCheckingExtension` compatibility with the dynamic DSL is validated iteratively — the runtime bindings of `g` and `graph` are the primary tension point and may require custom type resolution hints rather than strict static analysis.

Phase 03 Deliverable

`HugeScriptTraversal` modified to use `HugeGraph`-managed `EngineFactory` instead of `SingleGremlinScriptEngineManager`. Custom `GremlinGroovyScriptEngine` factory established with `CompilerConfiguration` injection. `SecureASTCustomizer` and `TypeCheckingExtension` active at compile time with allowlist validated against real traversal workloads. `GremlinSecurityPolicy` class extracted from `HugeSecurityManager` with clear delegation boundary. Gremlin checks disabled when sandbox is active. Backend protections for MySQL, HBase, Raft, and SofaRPC retained via `HugeSecurityManager` as runtime fallback. Sandbox policy configurable via `hugegraph.properties` without code changes.

Phase 03 Deliverable

Spring Boot in PD and Store is 2.5.14, not Boot 3. Boot 3 migration is out of scope — it implies Jakarta namespace migration and is a separate project. gRPC 1.39.0 in PD has a known Java 17 compatibility gap (1.42+ required). The run-tinkerpop-test.sh script referenced in CI does not exist in the repository. twitter-2010 SNAP dataset (~1.47B edges) benchmark infrastructure does not exist — schema, loader config, and REST driver must be written as part of this phase

The final phase validates the complete upgraded stack, migrates PD and Store to Java 17, stabilizes the test suite, and produces the performance comparison report. The scope is grounded in what the repository actually contains: Spring Boot 2.5.14 in both PD and Store, gRPC 1.39.0 in PD with a confirmed Java 17 compatibility gap, TinkerPop compliance tests gated to release branches only, and no existing graph benchmark harness. All of these constraints shape the actual work.

● PD & Store Module Migration

PD and Store currently run Spring Boot 2.5.14, which does not officially support Java 17 without workarounds. The migration path is Spring Boot 2.7.x, which is the last 2.x release with official Java 17 support, rather than Boot 3, which would require a full Jakarta namespace migration and is out of scope. The more concrete blocker is gRPC 1.39.0 in PD, which has known internal JDK API access issues on Java 17. Upgrading to gRPC 1.42+ is required before PD can run reliably on Java 17. RocksDB JNI 7.7.3 in Store is validated for binary compatibility through integration testing. All three startup scripts enforce EXPECT_JDK_VERSION=11 with no upper bound, so Java 17 passes version checks as-is, but separate RocksDB data paths must be configured for Java 11 baseline and Java 17 test environments to prevent forward-compatibility issues with RocksDB data files.

● Test Suite Stabilization

The run-tinkerpop-test.sh script referenced in the CI workflow does not exist in the repository. TinkerPop compliance tests are defined in hugegraph-test via ProcessStandardTest and StructureStandardTest, activated through Maven profiles, and are only executed on release-* or test-* branches. The compliance test infrastructure is restored by writing the missing script and enabling it on the Java 17 CI matrix. Unit test failures from the dependency upgrades are resolved systematically, prioritizing the Gremlin execution path, CRUD operations, and serialization behavior. Integration tests run against a live server instance to surface runtime-only failures including reflective access issues that do not appear in compilation.

● Regression Verification

Semantic changes across TinkerPop 3.6–3.8 are validated against the actual upgrade documentation rather than assumed. The confirmed high-risk coupling is TraversalUtil.extractOrder(), which calls orderStep.getComparators() and accesses .getValue1() on the returned Javatuples Pair to extract the ElementValueComparator. If TinkerPop 3.8 changes the return type of getComparators(), this is a compile-time break that must be addressed first. The choose step behavioral change and any other semantic differences are verified by cross-checking the TinkerPop 3.6, 3.7, and 3.8 upgrade documentation against HugeGraph's actual traversal usage before claiming regression coverage.

● Performance Benchmark Construction

The existing benchmark infrastructure is limited to a custom PerfExampleBase harness testing a hardcoded toy graph (100 edges), rather than a large-dataset harness. Furthermore, the standard TinkerPop performance suite is entirely blacklisted due to a vertex-label deletion failure above 65,536 vertices — rendering it unusable at the twitter-2010 scale (41M+ nodes). The benchmark infrastructure for the twitter-2010 SNAP dataset (~1.47B edges) is written as part of this phase. Deliverables include the schema definition, hugegraph-loader configuration, and a custom KoutTraverser REST driver targeting /traversers/kout at depths 1 and 2. The Java 11 baseline and Java 17 target run against separate RocksDB data paths, capturing latency, throughput, and GC pause metrics to explicitly measure the gains of ZGC and Shenandoah GC.

// benchmark construction — what must be built

Data Loader

twitter-2010 SNAP dataset loader (~1.47B edges) — schema, loader config, and KoutTraverser REST driver must be written

Workloads

Single-hop, multi-hop, concurrent write definitions — must be defined

Baseline

Java 11 + TinkerPop 3.5.1 — reference run

Target

Java 17 + TinkerPop 3.8 — separate RocksDB data path

Dependency	Current State	Action
Spring Boot (PD + Store)	2.5.14 — does not officially support Java 17. Boot 3 migration is out of scope (Jakarta namespace shift).	UPGRADE TO 2.7.x
gRPC (PD)	1.39.0 — known internal JDK API access issues on Java 17. 1.42+ required for reliable Java 17 operation.	UPGRADE TO 1.42+
gRPC (Server)	1.47.0 — already within Java 17 compatible range.	VALIDATE
RocksDB JNI (Store)	7.7.3 — ships native binaries. Java 17 JNI boundary validated through integration testing. Separate data paths required for baseline vs target runs.	VALIDATE
run-tinkerpop-test.sh	Script referenced in CI workflow but does not exist in the repository. TinkerPop compliance tests are effectively not running on any CI branch currently.	VALIDATE

Phase 04 Deliverable

PD and Store migrated to Java 17 with Spring Boot upgraded to 2.7.x and gRPC in PD upgraded to 1.42+. TinkerPop compliance test script restored and running against Java 17 CI matrix. Unit and integration tests passing. TraversalUtil OrderGlobalStep API coupling verified against TinkerPop 3.8 changelog. twitter-2010 SNAP dataset (~1.47B edges) benchmark executed against Java 11 baseline and Java 17 target with separate RocksDB data paths. Performance report produced with latency, throughput, and GC pause comparison under ZGC and Shenandoah GC.

Schedule & Timeline

HugeGraph Query Engine Upgrade & Adaptation

// PROJECT TIMELINE

Time Period	Timeline	Action
BONDING Community Bonding Phase	May 1 – May 24	- Interact with mentors, finalize project workflow, and establish development environment with Java 11 baseline - Validate incremental upgrade path (3.5.1 → 3.6 → 3.7 → 3.8) with mentors and confirm GraphBinary custom type registration feasibility - Finalize twitter-2010 benchmark design, schema, and workload definitions with mentors
PHASE 1 Decoupling & Core Upgrade	May 25 – Jun 21	- Replace all shaded Jackson and Kryo imports; declare direct Maven dependencies - Migrate Gryo IO layer to GraphBinary; replace MetricManager with Dropwizard singleton - Fix GroovyTranslator, GremlinExecutor, CypherOpProcessor, CredentialGraphTokens, and remaining breaking changes; compile clean at each incremental TinkerPop step
PHASE 2 Java 17 Runtime & Environment	Jun 22 – Jul 5	- Upgrade maven-compiler-plugin and surefire; migrate hugegraph-commons to Java 17 independently - Replace PowerMock with Mockito; align Jackson versions under a single BOM - Validate internal JDK API usages under Java 17; update all Dockerfiles and CI pipelines to eclipse-temurin:17
MIDTERM EVAL Midterm Evaluation	Jul 6 – Jul 10	Submit Phase 1 and Phase 2 deliverables – server compiling and running cleanly on TinkerPop 3.8 + Java 17

PHASE 3 Security Model Refactor	Jul 6 – Aug 1	• Replace SingleGremlinScriptEngineManager with a HugeGraph-managed EngineFactory injecting CompilerConfiguration • Attach SecureASTCustomizer, TypeCheckingExtension, and TimedInterruptCustomizer to the sandbox stack • Extract GremlinSecurityPolicy from HugeSecurityManager; expose sandbox config in hugegraph.properties
PHASE 4 Testing & Benchmarking	Aug 2 – Aug 16	• Migrate PD and Store to Spring Boot 2.7.x and gRPC 1.42+; restore missing run-tinkerpop-test.sh • write twitter-2010 benchmark infrastructure and execute Java 11 vs Java 17 comparison. • Run Java 11 vs Java 17 benchmarks on separate RocksDB data paths; produce performance comparison report
FINAL SUBMISSION Final Evaluation	Aug 17 – Aug 24	• Submit all deliverables, complete documentation, and final refactoring • Mentors submit final evaluations by Aug 24–31

Availability

I plan to dedicate 35–45 hours per week to the project throughout the coding period. I will be most active on weekdays from 9 AM to 9 PM IST (UTC +05:30). I will update mentors daily and demonstrate progress through weekly syncs or async recordings. Any academic commitments will be communicated to mentors at least two weeks in advance.

I am 100% dedicated to Apache Hugegraph and have no plans whatsoever to submit a proposal to any other Organization.